Trabalho Final — Ferramenta de Administração Remota via Sockets

O trabalho final consiste em construir uma **ferramenta de administração remota (RAT — *Remote Administration Tool*)** cliente-servidor, operada por sockets TCP/IP, capaz de enviar comandos a uma máquina remota e de controlar seu mouse e teclado — o mesmo princípio de VNC, TeamViewer e RDP. Todo o uso é **consentido e supervisionado**: a execução acontece apenas em máquinas próprias ou do laboratório, com indicação visível no agente de que ele está sendo controlado, e sem qualquer recurso de ocultação, persistência ou evasão. É esse recorte — transparência e consentimento — que separa a ferramenta de estudo de um *malware*, e é justamente ele que será avaliado.

1. Apresentação e contexto

Ferramentas de administração remota exercitam, num só projeto, quase todos os subsistemas estudados na disciplina: processos e *threads*, comunicação entre máquinas por sockets, serialização de dados, o subsistema de entrada (mouse e teclado) e o de saída (tela), além de concorrência e sincronização.

Historicamente, o conceito ficou associado a programas como o **Back Orifice** (1998), apresentado à época como ferramenta de administração remota, mas usado sobretudo de forma malévola por operar às escondidas, sem consentimento do usuário da máquina. A mesma funcionalidade técnica — controlar um computador pela rede — sustenta hoje produtos legítimos e onipresentes: **VNC**, **RDP** (Área de Trabalho Remota do Windows), **TeamViewer**, **AnyDesk**, **SSH** e as ferramentas de automação de frota como **Ansible**.

A diferença entre as duas famílias **não está no código de rede nem na injeção de entrada**, que são idênticos; está em três escolhas de projeto: **consentimento** (o dono da máquina sabe e autoriza), **transparência** (a operação é visível) e **ausência de ocultação** (o programa não se esconde nem se perpetua). Este trabalho constrói a família legítima e usa esse contraste como objeto de estudo explícito.

O Back Orifice entra aqui apenas como **referência histórica e conceitual**; o produto do trabalho é uma ferramenta de controle remoto assumidamente visível, no espírito de um VNC didático.

2. Objetivos de aprendizagem

Ao concluir o trabalho, o aluno deve ser capaz de:

1. Implementar comunicação em rede com **sockets TCP/IP**, tratando conexão, fragmentação de mensagens e desconexão.
2. Projetar um **protocolo de aplicação** próprio: formato de mensagem, enquadramento (*framing*), tipos de comando e respostas.
3. Aplicar **serialização** (JSON, Protobuf ou binário) para transportar comandos e dados estruturados.
4. Usar **threads** e sincronização para atender rede e interface sem travar a aplicação.
5. Operar as **APIs de entrada do sistema operacional** para mover o cursor, clicar e injetar teclas.

6. Executar **processos filhos** e capturar sua saída padrão para a execução remota de comandos.
7. Discutir e implementar **autenticação e cifragem** (TLS) do canal de controle.
8. Analisar criticamente a fronteira **ética e legal** entre administração remota legítima e *software* malicioso.

3. Enquadramento ético, legal e de segurança

Estas regras fazem parte da especificação e valem como requisito de aprovação: um trabalho que as viole está reprovado, por melhor que seja o código. Elas também são o que mantém o projeto do lado legítimo da linha.

Regras invioláveis (o que o projeto DEVE ser):

- **Ambiente controlado.** Execução apenas em máquinas do próprio grupo ou do laboratório da universidade, em rede isolada ou local. Nunca em equipamento de terceiros.
- **Consentimento explícito.** O agente só roda em uma máquina cujo dono autorizou. Para a defesa, todas as máquinas são do grupo.
- **Indicador visível.** Enquanto controlado, o agente mostra um aviso permanente e indispensável na tela (janela ou ícone de bandeja): *"Esta máquina está sob controle remoto — Trabalho de SO"*, com o endereço do controlador conectado.
- **Início manual.** O agente é iniciado à mão pelo operador na máquina; nada de execução automática silenciosa.
- **Desligamento local.** Um atalho na própria máquina do agente encerra a sessão e o programa a qualquer momento (*kill switch*).
- **Registro (log).** Toda conexão e todo comando recebido são gravados em arquivo de log legível no agente.

Fora do escopo (o que o projeto NÃO PODE ter): estes itens, característicos de *malware*, estão proibidos e sua presença reprova o trabalho.

- Ocultação de processo, janela ou tráfego; qualquer tentativa de passar despercebido ao usuário.
- Persistência: entradas de inicialização automática, serviços, tarefas agendadas, alteração de registro para sobreviver a reinicializações.
- Evasão de antivírus, *ofuscação* com esse fim, empacotamento para dificultar análise.
- Propagação: qualquer mecanismo de autocópia, infecção ou disseminação pela rede.
- Disfarce: fingir ser outro programa legítimo, ícone ou nome enganoso.
- Coleta sorrateira de credenciais, *keylogging* persistente em segundo plano fora de uma sessão visível, exfiltração de dados pessoais.

4. Visão geral do sistema e arquitetura

O sistema tem dois programas que conversam por um socket TCP. Os nomes clássicos *master/slave* mapeiam assim:

- **Controlador (console do operador):** a aplicação com interface onde o operador digita comandos e vê respostas. No padrão de controle remoto, é quem **inicia a conexão** (cliente TCP). Corresponde ao "master".
- **Agente (máquina administrada):** o programa que roda na máquina a ser controlada, **escuta** numa porta, recebe comandos, executa-os localmente e devolve

o resultado. É o servidor TCP e corresponde ao “slave”. É ele que exibe o indicador visível da Seção 3.

Um controlador pode administrar vários agentes; cada agente atende um controlador por vez. A separação de papéis é lógica: os dois lados usam a mesma biblioteca de protocolo.

Fluxo de uma sessão:

1. O agente é iniciado na máquina administrada, exibe o aviso de controle remoto e passa a escutar (ex.: porta 5000).
2. O controlador conecta ao endereço do agente e **autentica-se** (segredo compartilhado ou certificado).
3. Estabelecido o canal (de preferência sobre **TLS**), o operador envia comandos; cada comando vira uma mensagem do protocolo.
4. O agente executa o comando no sistema operacional e responde com resultado, saída ou erro.
5. O operador encerra a sessão, ou o agente é desligado localmente pelo *kill switch*.



Arquitetura: Controlador, Agente e subsistemas do SO

O Controlador (cliente) abre a conexão; o Agente (servidor) executa cada comando e responde, e cada capacidade aciona um subsistema do sistema operacional.

5. Requisitos funcionais

Divididos em obrigatórios (nota base) e opcionais (pontos extras). Cada grupo exige um comando no protocolo e seu tratamento nos dois lados.

#	Capacidade	O que faz	Nível
RF1	Conexão e sessão	Agente escuta; controlador conecta, autentica e mantém a sessão	Obrigatório
RF2	Execução de comando de shell	Rodar um comando do SO no agente e devolver saída padrão e código de saída	Obrigatório
RF3	Informações do sistema	Agente devolve SO, usuário, hostname, uso de CPU/memória e discos	Obrigatório
RF4	Listar e navegar arquivos	Listar diretório, ver atributos (liga com a unidade de sistemas de arquivos)	Obrigatório
RF5	Transferência de	Enviar e baixar arquivos entre controlador e	Obrigatório

#	Capacidade	O que faz	Nível
	arquivos	agente, com verificação de integridade	
RF6	Mover o mouse	Controlador posiciona o cursor na tela do agente (absoluto e relativo)	Obrigatório
RF7	Cliques e rolagem	Botões esquerdo/direito/meio e *scroll*	Obrigatório
RF8	Injetar teclado	Digitar texto e enviar combinações de teclas no agente	Obrigatório
RF9	Captura de tela	Agente envia uma imagem da tela sob demanda	Opcional
RF10	Tela contínua (mini-VNC)	Fluxo de imagens periódicas para controle visual em tempo quase real	Opcional
RF11	Lista de processos	Listar processos e encerrar um processo por PID	Opcional
RF12	Múltiplos agentes	Um controlador gerencia vários agentes simultaneamente	Opcional

Observação sobre RF8/RF9: a captura de tela e a injeção de teclado existem **somente dentro de uma sessão visível e consentida**; registrar teclas em segundo plano fora disso cai na lista proibida da Seção 3.

6. Requisitos não funcionais

- **Segurança do canal:** autenticação antes de qualquer comando e, de preferência, cifragem TLS. Sem autenticação, o agente recusa a conexão.
- **Robustez:** o agente não pode travar com mensagem malformada nem com queda do controlador; a sessão encerra de forma limpa.
- **Concorrência:** a rede roda em *thread* separada da interface, para que receber dados não congele a tela.
- **Portabilidade:** idealmente o agente roda em Windows e Linux; se o grupo optar por um só, deve justificar e isolar a parte dependente de SO.
- **Desempenho:** a latência de um comando simples deve ser perceptível como "imediata" na rede local; a tela contínua (RF10) deve manter taxa de quadros utilizável.
- **Observabilidade:** log no agente e no controlador, com data/hora, origem e comando.
- **Usabilidade:** interface do controlador clara, com lista de agentes e área de resposta.

7. Protocolo de comunicação

TCP entrega um **fluxo de bytes**, não mensagens: dois comandos podem chegar grudados ou um comando partido em dois recv. O grupo precisa definir o **enquadramento** (*framing*). A abordagem recomendada: cada mensagem é um JSON precedido de um cabeçalho de 4 bytes com o tamanho do corpo (*length-prefixing*).

[4 bytes: tamanho N][N bytes: corpo JSON em UTF-8]

Envelope da mensagem (exemplo em JSON):

```
{
  "id": "uuid-da-requisicao",
  "tipo": "comando | resposta | evento",
  "acao": "EXEC | SYSINFO | LS | GET | PUT | MOUSE_MOVE | MOUSE_CLICK | KEY |
```

```
SCREENSHOT",
  "args": { },
  "timestamp": "2026-10-28T14:30:00Z"
}
```

Exemplos de ações e argumentos:

Ação	args principais	Resposta
EXEC	{ "cmd": "ls -la" }	saída padrão, saída de erro, código de retorno
MOUSE_MOVE	{ "x": 840, "y": 410, "abs": true }	ok ou erro
MOUSE_CLICK	{ "botao": "esq", "duplo": false }	ok
KEY	{ "texto": "ola" } ou { "combo": ["ctrl","c"] }	ok
GET	{ "caminho": "/home/ana/log.txt" }	arquivo em blocos (*chunks*)
SCREENSHOT	{ "formato": "png", "qualidade": 70 }	imagem em base64 ou blocos binários

Decisões de protocolo que o grupo deve documentar: síncrono x assíncrono (correlacionar resposta pelo id); texto (JSON) x binário (Protobuf/*MessagePack*); como fatiar arquivos e imagens grandes; e como sinalizar e tratar erros. A fase de autenticação (*handshake*) vem antes de qualquer ação: o controlador prova o segredo e o agente aceita ou derruba a conexão.

8. Componentes técnicos e ligação com conceitos de SO

Cada capacidade do projeto aciona um subsistema estudado na disciplina — é isso que torna o trabalho um fechamento do semestre, e não só um exercício de rede.

Capacidade do projeto	Subsistema / conceito de SO	O que exercita
Sockets TCP, *framing*	Comunicação entre processos e em rede	Pilha de I/O, portas, fluxo de bytes, *buffering*
Rede + interface sem travar	Threads e sincronização	Concorrência, regiões críticas, filas
EXEC de comando	Criação de processos	fork/exec, CreateProcess, *pipes* de saída
Mover mouse e injetar teclas	Subsistema de entrada	APIs de injeção: SendInput, uinput, XTest
Captura de tela	Subsistema de saída / vídeo	*Framebuffer*, composição, GDI/X11
Listar arquivos, transferir	Sistema de arquivos	Diretórios, metadados, descritores, I/O de arquivo
Info de sistema	Chamadas de sistema	/proc, WMI, APIs de SO
TLS e autenticação	Segurança	Criptografia de canal, controle de acesso

Injeção de entrada — por API, nunca por hardware falso. Mover o cursor e digitar são feitos pelas **mesmas APIs que a tecnologia assistiva usa**, oferecidas pelo próprio SO:

- **Windows:** função SendInput (Win32).
- **Linux (X11):** extensão XTest; em Wayland, via libei/portais.
- **Linux (baixo nível):** dispositivo virtual uinput.
- **Bibliotecas de alto nível:** java.awt.Robot (Java), pyautogui ou pynput (Python), robotjs (Node).

Essas APIs são públicas e documentadas justamente por serem o mecanismo legítimo de automação e acessibilidade — o que muda num *malware* não é a API, é o consentimento e a visibilidade.

9. Tecnologias sugeridas

O grupo escolhe a linguagem; abaixo, pilhas que cobrem bem todos os requisitos. Qualquer uma é aceita desde que atenda ao protocolo e às regras da Seção 3.

Linguagem	Sockets	Entrada (mouse/teclado)	Tela	TLS
Python	socket, selectors, asyncio	pyautogui, pynput	mss, Pillow	ssl
Java	java.net.Socket	java.awt.Robot	Robot.createScreenCapture	SSLSocket
C# / .NET	TcpListener, TcpClient	SendInput (P/Invoke)	Graphics.CopyFromScreen	SslStream
Node.js	net	robotjs, nut.js	screenshot-desktop	tls
C / C++	*sockets* POSIX / Winsock	SendInput, uinput	GDI / X11	OpenSSL

Recomendação para a maioria dos grupos: Python pelo equilíbrio entre bibliotecas prontas (pynput, mss) e código de rede legível; ou Java, se a turma já usa Robot em Desenvolvimento. Grupos que queiram sentir o SO de perto podem fazer o agente em C com uinput/SendInput e o controlador em algo mais alto nível.

10. Etapas de desenvolvimento

Sugestão de evolução incremental; cada etapa termina com algo que funciona e pode ser testado.

1. **Eco por socket.** Controlador conecta, manda texto, agente devolve. Resolve conexão e *framing*.
2. **Protocolo e EXEC.** Envelope JSON com id/acao; implementar RF1–RF3 (sessão, comando de shell, info do sistema).
3. **Arquivos.** RF4 e RF5: listar diretório e transferir arquivos em blocos, com verificação de integridade.
4. **Entrada.** RF6–RF8: mover mouse, clicar, injetar teclado pela API do SO. Entra o indicador visível.
5. **Segurança.** Autenticação no *handshake* e TLS no canal; *kill switch* e logs.
6. **Interface e opcionais.** Interface do controlador, captura/tela contínua (RF9–RF10), múltiplos agentes (RF12).
7. **Testes e documentação.** Testes em rede local, roteiro de demonstração e relatório.

Divisão em dupla ou trio: um cuida do núcleo de rede/protocolo, outro das capacidades do agente (entrada, tela, arquivos), outro da interface do controlador e da segurança. Combinem o protocolo cedo — é o contrato entre os dois lados.

11. Entregáveis

- Código-fonte do controlador e do agente, em repositório Git, com README de compilação e execução.
- Especificação do protocolo (ações, formato das mensagens, *handshake*).
- Relatório técnico (6 a 12 páginas): arquitetura, decisões de projeto, ligação com os conceitos de SO e análise ética/legal da Seção 3.
- Vídeo ou demonstração ao vivo (até 5 min) em duas máquinas do laboratório, mostrando o indicador visível no agente.
- Roteiro de teste usado na demonstração.

12. Critérios de avaliação

Total de 100 pontos; os opcionais valem como bônus até o teto. O cumprimento das regras da Seção 3 é condição de aprovação, não um critério pontuado: sua violação zera o trabalho.

Critério	Pontos
Comunicação por sockets e protocolo bem definido (RF1)	20
Execução remota de comandos e info do sistema (RF2–RF3)	15
Arquivos: listagem e transferência (RF4–RF5)	15
Controle de mouse e teclado (RF6–RF8)	20
Segurança: autenticação e TLS, logs, *kill switch*	10
Concorrência e robustez	5
Relatório e análise ética/legal	10
Demonstração e organização do código	5
Base	**100**
Bônus: captura/tela contínua, múltiplos agentes, processos (RF9–RF12)	até +15

Condição eliminatória: qualquer item da lista proibida da Seção 3 (ocultação, persistência, evasão, propagação, disfarce, coleta sorrateira) reprova o trabalho, independentemente da nota técnica.

13. Desafios e extensões opcionais

Para grupos que terminarem o núcleo e quiserem aprofundar:

- **Mini-VNC:** fluxo de tela contínuo com envio só das regiões que mudaram (*dirty rectangles*) e compressão.
- **Descoberta na rede:** o controlador encontra agentes na LAN por *broadcast*/mDNS, em vez de IP digitado.
- **Reconexão automática** e fila de comandos resiliente a quedas.
- **Transferência eficiente:** blocos com confirmação, retomada e *hash* por bloco.
- **Painel multiagente:** uma interface que mostra todos os agentes conectados e seu estado.

- **Análise defensiva:** documentar como um antivírus ou EDR detectaria a ferramenta e por que, discutindo os sinais que distinguem uso legítimo de abuso — o lado azul (*blue team*) da mesma moeda.

14. Referências

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. 10. ed. LTC, 2022 — capítulos de processos, threads e I/O.
- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. Pearson, 2016.
- STEVENS, W. R.; FENNER, B.; RUDOFF, A. M. *Unix Network Programming, Vol. 1: The Sockets Networking API*. 3. ed. Addison-Wesley.
- BRASIL. **Lei nº 12.737/2012** (Lei Carolina Dieckmann) e art. 154-A do Código Penal; **Lei nº 13.709/2018** (LGPD).
- Documentação das APIs de entrada: Microsoft SendInput, X11 XTest, Linux uinput, java.awt.Robot, pynput.
- RFC 8446 — **TLS 1.3**, para a cifragem do canal.